CLOUD-02: AWS Integration

> **Series:** CLOUD — Cloud Provider Integrations | **Notebook:** 2 of 8 | **Created:** March 2026 | **Last Updated:** 08/12/2026

Overview

This notebook covers how to set up and optimize Dynatrace's AWS integration. You will learn about IAM role-based and key-based authentication, which AWS services are supported, how to query AWS entities and metrics, and strategies for managing API costs.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS or Managed with Grail enabled
Permissions	`metrics.read`, `entities.read`, `ReadConfig`, `WriteConfig`
AWS Account	With IAM permissions for Dynatrace integration
Connection	Clouds app direct connection (recommended) or Environment ActiveGate (classic)
Prior Knowledge	CLOUD-01 fundamentals

1. Authentication Methods

Dynatrace AWS integration is **role-based** today. Key-based (access key + secret) authentication was **removed for new credentials starting with Dynatrace 1.267**; existing key-based connections still function but cannot be created or rotated through the modern UI. New AWS connections must use IAM role assumption.

Current — IAM Role via CloudFormation (Clouds App)

This is the default and recommended path. The Clouds App onboarding wizard generates a CloudFormation template that creates the Dynatrace monitoring IAM role inside the customer's AWS account, configures the trust policy so Dynatrace SaaS can `sts:AssumeRole` into it, and registers the connection.

- **No long-lived AWS credentials are stored in Dynatrace** — only the role ARN and Dynatrace-side Platform Tokens
- **Trust is bound to the Dynatrace SaaS account** (and to an `ExternalId` to prevent the confused-deputy class of attack)
- **Cross-account monitoring** works via additional trust policies on each monitored account's role
- **Connection health** in the Clouds App reflects whether the SaaS platform was able to successfully assume the role

Legacy — Key-Based Authentication (Deprecated)

Pre-1.267 connections that used an IAM-user access key + secret continue to function but should be migrated:

- New tenants cannot create key-based connections
- Existing keys cannot be rotated through the new Clouds App flow
- IAM-user access keys are long-lived, manually rotated, and are a known credential-exposure risk

> **Migration:** if you still have key-based connections, plan their conversion to role-based via the Clouds App onboarding. Old connections will remain monitored but new services or regions should be added to a fresh role-based connection.

Separate Surface — AWS Connector for Workflows

The **AWS Connector** used by Workflow actions (the `aws_*` action set) is a different integration with its own credential model — it authenticates via OpenID Connect (OIDC) or cross-account role assumption. Its **legacy connection schema is deprecated effective March 31, 2026** — Workflow customers must migrate to the enhanced connection schema. This deprecation is scoped to the Workflows connector and does **not** affect Clouds App AWS connections used for telemetry ingestion.

IAM Permissions

The Dynatrace monitoring role uses a **scoped read-only policy** authored as part of the CloudFormation stack — not the AWS-managed `ReadOnlyAccess` policy. The exact policy is defined inside the nested template `da-aws-nested-integration.yaml` that the Clouds App wizard generates. Review this nested template as part of the security-review workflow described in §7.3.1 Path A.

> **Practical note:** the role does not need write permissions for monitoring. If your security team requires a custom least-privilege policy in place of the Dynatrace-generated one, derive it from the nested template and pin it via your IaC pipeline. Re-verify after each Dynatrace release that adds new monitored services — additional services usually need new `Describe*` / `List*` permissions.

> **Sources:** Create AWS connection via Settings (DT docs), Manage your AWS connections (DT docs), Set up AWS Connector for Workflows (DT docs) — March 31 2026 legacy-schema deprecation. **Derived:** the "1.267 removed key-based for new credentials" cutover is confirmed in DT release-notes summaries; existing-key behaviour and migration urgency are field-observed guidance, not a documented sunset date. **Softened:** the "review the nested template" recommendation is community / SE practice — the exact `da-aws-nested-integration.yaml` permission scope evolves per Dynatrace release.

2. Supported AWS Services

Dynatrace monitors 80+ AWS services. Key services include:

Category	Services
Compute	EC2, Lambda, ECS, EKS, Fargate, Elastic Beanstalk
Storage	S3, EBS, EFS, Glacier
Database	RDS, DynamoDB, ElastiCache, Redshift, Aurora
Networking	ELB/ALB/NLB, CloudFront, Route 53, API Gateway, VPC
Application	SQS, SNS, Step Functions, Kinesis, EventBridge
Container	ECS, EKS, Fargate
AI/ML	SageMaker, Bedrock

Service Monitoring Granularity

Service	Entity Type	Metric Interval	Key Metrics
EC2	`dt.entity.ec2_instance`	5 min	CPU, network, disk, status checks
Lambda	`dt.entity.aws_lambda_function`	1 min	Invocations, duration, errors, throttles

Service	Entity Type	Metric Interval	Key Metrics
RDS	`dt.entity.relational_database_service`	5 min	CPU, connections, read/write latency
S3	Custom device	1 day	Bucket size, object count, requests
ELB	`dt.entity.elastic_load_balancer`	1 min	Request count, latency, HTTP errors

Entity Type Notation

Two notation systems for AWS entities coexist in Dynatrace today:

Surface	Notation	Example
Classic entities API (`fetch dt.entity.*`)	Lowercase, underscored	`dt.entity.ec2_instance` , `dt.entity.aws_lambda_function`
Smartscape 2.0 / Grail (`smartscapeNodes`)	Uppercase, from CloudFormation resource types with `::` → `_`	`AWS_EC2_INSTANCE` , `AWS_LAMBDA_FUNCTION`

Both work today. The DQL examples in §4 and §5 show the classic form with the Smartscape 2.0 alternative commented out — pick the one that matches your tenant's preferred query style.

> **Sources:** All AWS cloud services (DT docs), AWS topology (DT docs) — verbatim: *"Smartscape node types for AWS follow the CloudFormation resource type notation, making all letters uppercase and substituting :: with _"*, AWS supported services API (DT docs). **Softened:** the "80+ services" count varies by Dynatrace release as new services are added — check the Hub or the all-services page for the current catalog rather than treating any fixed number as authoritative.

3. Enabling Service Monitoring

Connection Methods

Recommended: Clouds App (single AWS account)

The Clouds App is the modern onboarding path for SaaS tenants. It is **polling-based** (CloudWatch API per region) — for push-based metric ingestion, see §7.

1. Open the **Clouds app → Create Connection → AWS (New)**
2. Enter connection name, AWS Account ID, and the **monitored regions** (`us-east-1` is required for global AWS resources)
3. Pick a path:
   - **Recommended** — opinionated, immutable defaults; fastest onboarding
   - **Advanced (Fine-Grained)** — full control over metrics, logs, tags, and enrichment (best practice for production tenants)
4. Generate **Platform Tokens** (the wizard auto-creates the Service Users and mints two tokens — one for settings management, one for telemetry ingestion). **No classic API token is needed for the Clouds App.**
5. Deploy the CloudFormation stack:
   - If you have AWS access — open the CFN template via the direct link
   - If you don't — use **Copy Link** to hand the deployment URL to the AWS admin

AWS Organizations (multi-account)

For org-wide onboarding, use the **Settings app** (not the Clouds App) to create an organization-monitoring configuration. Requires an AWS Organizations delegated admin with permission to run StackSets in the management region. See the Onboard AWS Organizations docs.

Classic: Settings UI / ActiveGate polling

For Managed deployments or environments with strict network requirements, the classic Settings-based AWS connection is still supported (Settings → Cloud and virtualization → AWS).

Advanced-Path Configuration Surface

The Advanced path exposes fine-grained controls. Exact UI labels evolve — verify against your tenant.

Control	What it does
CloudWatch Metrics scope	Choose ingestion mode per service: **Recommended** (DT-curated essentials), **Recommended + Custom** (essentials plus tenant-specified metric keys / dimensions), or **Auto-Discovery** (poll all available metrics, then narrow down)
CloudWatch Logs	Enable log ingestion per region; toggleable post-onboard by updating the CFN stack. Delivered via Kinesis Data Firehose subscriptions — see CLOUD-07. Note the scope: this control covers logs flowing *through CloudWatch*. Logs **already resident in S3** take a separate **direct-ingestion** path (a Dynatrace-maintained Lambda forwarder, forthcoming / rolling out with SaaS 1.344) that bypasses CloudWatch and Firehose entirely — see CLOUD-07 §3
Tag Enrichment	All AWS tags are collected by default. This control selects which tags propagate to logs / metrics / spans / events as dimensions
Tag-Based Filtering	Include / exclude resources from monitoring based on AWS tag predicates — scope to specific workloads or teams
Dynatrace Attribute Enrichment	Map AWS tag values onto Dynatrace native attributes such as `dt.security_context`, `dt.cost.product`, `dt.cost.costcenter` — enables IAM segments and cost-allocation dashboards driven by AWS-side metadata. **Enrichment configured inside a connection is moving to a central ingest-time configuration** — see the note below
EventBridge events	Capture AWS events via EventBridge API destinations (region-availability dependent)

> **In community practice:** start with Recommended; promote to Recommended + Custom when you know the specific metric keys you need; reserve Auto-Discovery for discovery / scoping exercises, then trim back. Auto-Discovery left on indefinitely inflates DDU and CloudWatch API cost.
>
> **Where enrichment lives is changing.** Tag and attribute enrichment rules configured *inside* a cloud connection are being replaced by a **central ingest-time enrichment configuration**, so the same rules are defined once and apply across connections rather than being embedded in each. The readiness scan reports connections still carrying embedded rules, along with how many each holds.
>
> This does not change what to enable — the Advanced path remains the right posture for production tenants, and `dt.security_context` / `dt.cost.*` mapping is still what

> makes IAM segments and cost allocation work. What changes is *where you define it*. If you are configuring a connection now, expect to recreate its enrichment rules centrally later; connections carrying no embedded rules simply opt in. Classic AWS connections themselves are recreated rather than converted when moving to the latest Dynatrace — see CLOUD-01 §2.

Service Selection Strategy

Rather than enabling all 80+ services, start with the services your applications depend on:

Tier	Services to Enable	Rationale
Tier 1 (Always)	EC2, ELB, RDS, Lambda	Core infrastructure
Tier 2 (Common)	S3, SQS, SNS, DynamoDB, ECS/EKS	Application dependencies
Tier 3 (As Needed)	CloudFront, Route 53, Kinesis, Step Functions	Edge/workflow services
Tier 4 (Selective)	SageMaker, Bedrock, IoT	Specialized workloads

Discovery Questions (Before Onboarding)

- In which AWS regions are the customer's resources and signals located?
- Do they have an existing tagging strategy? Which tags should propagate as Dynatrace dimensions? Which tags map to `dt.security_context` / `dt.cost.*`?
- Single AWS account, or multiple accounts via AWS Organizations?
- Does the Dynatrace administrator have AWS console access, or will deployment be handed off?

Multi-Account Patterns at Scale

Beyond the single-account / AWS Organizations choice at onboarding, three layered patterns emerge in practice for tenants running many accounts:

Pattern	When to use	Key mechanic
One connection per AWS Organization	Single org, <50 accounts, shared monitoring posture	Delegated admin role + Org-wide StackSets push the monitoring role into all member accounts
One connection per Organizational Unit (OU)	Mixed regulated/non-regulated workloads, distinct DT tenants per business unit	StackSet target = OU rather than the whole org; each Clouds App connection scopes to its OU's accounts
Hybrid: org connection for telemetry + per-account connection for sensitive workloads	A small subset of accounts need stricter scope (different IAM policy, different log destination, different region set)	Org connection covers the long tail; targeted single-account connections override for the exception cases

> **In community practice**, the org-connection-with-OU-targeted-StackSets pattern reaches an acceptable rollout cadence for tenants in the 50–500 account range — each new account joining an OU gets the monitoring role within a single Org reconcile cycle. Above ~500 accounts, expect StackSet drift to become a regular operational concern; tenants that scale further typically split into per-OU connections.

AWS Cost Categories ↔ `dt.cost.product` mapping. If your AWS organization uses AWS Cost Categories for chargeback, mirror the Cost Category dimension into `dt.cost.product` via Dynatrace Attribute Enrichment in §3's Advanced path. Result: a single dimension joins AWS billing data to Dynatrace cost and capacity dashboards without per-team mapping logic on either side. Field-observed; verify the dimension naming with your FinOps team before rolling out.

EventBridge — Ingesting AWS Events into Davis

The Clouds App's **EventBridge** option (Advanced path) registers an EventBridge API destination pointing at the Dynatrace event-ingest endpoint. Once configured, AWS service events (EC2 state changes, EBS volume modifications, RDS failovers, GuardDuty findings, etc.) flow into Dynatrace as bizevents and can drive Davis problems, trigger Workflows, or feed dashboards.

Step	What happens
1. Enable in Clouds App	Advanced path → EventBridge option (region-availability dependent)
2. EventBridge rule	The wizard creates rule(s) matching the AWS event patterns you select
3. API destination	Routes matched events to the Dynatrace event-ingest endpoint with the connection's Platform Token
4. Ingest into Grail	Events land as `bizevent` records, queryable via `fetch bizevents`
5. Davis / Workflows	Use the events as Davis problem triggers or in Workflow actions

> **In community practice**, EventBridge ingestion is most valuable for **security and operational signals AWS already produces** (GuardDuty, Inspector, Health Dashboard, AutoScaling) rather than for application telemetry — application events are better generated directly by your code and pushed to OneAgent or the Business Events API.
>
> **Sources:** Create AWS connection via Settings (DT docs), Monitor AWS with CloudWatch metrics (DT docs), Onboard AWS Organizations (DT docs), AWS Cost Categories (AWS docs), EventBridge API destinations (AWS docs). **Derived:** the Advanced-path control surface table synthesizes the public docs with field-observed Clouds App UI; verify exact labels in your tenant. The multi-account pattern matrix and the OU-StackSet rollout cadence are field-observed — no DT-published guidance ranks them. **Softened:** the Recommended → Recommended+Custom → trim-from-Auto-Discovery promotion sequence, the org-vs-per-OU break-point at ~500 accounts, and the "EventBridge for security signals, OneAgent for app telemetry" framing are community / SE guidance, not documented best practices.

4. Querying AWS Entities

List All EC2 Instances

Governance Pattern — Find Resources Missing a Required Tag

A common operational pattern: identify AWS resources that haven't been onboarded into your tagging governance scheme. This becomes a recurring dashboard tile or a Davis-triggered Workflow.

> **Sources:** DQL fetch reference (DT docs), Entity model (DT docs). Entity types verified at All AWS cloud services (DT docs).

```
// List all EC2 instances with their names and instance types
//
// Corrected 08/12/2026: awsAvailabilityZone DOES NOT EXIST on
dt.entity.ec2_instance and made the
// whole cell fail with FIELD_DOES_NOT_EXIST. The model carries awsVpcName /
awsInstanceId / amiId /
// awsSecurityGroup / publicIp / localIp — there is no availability-zone
attribute. Confirm with:
//   fetch dt.semantic_dictionary.models | filter name ==
"dt.entity.ec2_instance" | fields fields
fetch dt.entity.ec2_instance, from:-7d
| fieldsKeep id, entity.name, tags, awsInstanceType, awsVpcName, awsInstanceId
| sort entity.name asc
| limit 20

// Time range required (corrected 08/12/2026): dt.entity.* is an event-
LOOKBACK view — it returns
// only entities SEEN in the query window, not the standing inventory. Without
an explicit from:
// this counted 5 of 13 EC2 instances against the notebook default window and
looked correct.
// Smartscape note (dt.entity.* is deprecated but still functional): this
cloud resource type
// (EC2 / Azure VM / RDS / Azure SQL / Azure Web App) is not modeled as a
Smartscape node — such
// hosts surface as smartscapeNodes "HOST" with cloud.provider and
aws.*/azure.* fields. Keep the
// classic query above for the cloud-resource inventory.
```

Count EC2 Instances by Instance Type

```
// Count EC2 instances grouped by instance type
fetch dt.entity.ec2_instance, from:-7d
| summarize instance_count = count(), by:{awsInstanceType}
| sort instance_count desc

// Time range required (corrected 08/12/2026): dt.entity.* is an event-
LOOKBACK view — it returns
// only entities SEEN in the query window, not the standing inventory. Without
an explicit from:
// this counted 5 of 13 EC2 instances against the notebook default window and
looked correct.
// Smartscape note (dt.entity.* is deprecated but still functional): this
cloud resource type
// (EC2 / Azure VM / RDS / Azure SQL / Azure Web App) is not modeled as a
Smartscape node — such
// hosts surface as smartscapeNodes "HOST" with cloud.provider and
aws.*/azure.* fields. Keep the
// classic query above for the cloud-resource inventory.
```

List Lambda Functions

```
// List all monitored Lambda functions
//
// Corrected 08/12/2026: the runtime attribute is awsRuntime, not awsLambdaFunctionRuntime — the
// latter does not exist and failed the cell with FIELD_DOES_NOT_EXIST.
fetch dt.entity.aws_lambda_function, from:-7d
| fieldsKeep id, entity.name, tags, awsRuntime, awsMemorySize, awsTimeout
| sort entity.name asc
| limit 20

// Time range required (corrected 08/12/2026): dt.entity.* is an event-LOOKBACK view — it returns
// only entities SEEN in the query window, not the standing inventory. Without an explicit from:
// this counted 5 of 13 EC2 instances against the notebook default window and looked correct.

// Smartscape note (dt.entity.* is deprecated but still functional): AWS Lambda functions ARE
// modeled on Smartscape as smartscapeNodes "AWS_LAMBDA_FUNCTION", but the classic aws* attribute
// fields differ there — inspect smartscapeNodes "AWS_LAMBDA_FUNCTION" | limit 1 for the node
// field names. Keep the classic query above until the fields are mapped.
```

List RDS Instances

```
// List all monitored RDS instances
fetch dt.entity.relational_database_service, from:-30d
| fieldsKeep id, entity.name, tags
| sort entity.name asc
| limit 20

// Returns no rows unless the AWS integration has RDS enabled — an empty result here means no RDS
// instance was seen in the window, NOT that the query is wrong. Distinguish the two by widening
// the window before concluding anything.

// Smartscape note (dt.entity.* is deprecated but still functional): this cloud resource type
// (EC2 / Azure VM / RDS / Azure SQL / Azure Web App) is not modeled as a Smartscape node — such
// hosts surface as smartscapeNodes "HOST" with cloud.provider and aws.*/azure.* fields. Keep the
// classic query above for the cloud-resource inventory.
```

```
// Find EC2 instances missing the required cost-allocation tag
//
// Corrected 08/12/2026. Two defects: (1) tags is an ARRAY of "key:value"
strings, not a map, so
// tags[`dt.cost.product`] failed with INNER_FIELD_OF_FIELD_DOES_NOT_EXIST;
(2) the null-safe half
// matters — an entity with NO tags at all has tags == null, and iAny(...)
over null yields null,
// which `not` leaves null and the filter silently drops. Without the isNull()
arm this reported
// 0 untagged instances while all 13 were in fact untagged.
fetch dt.entity.ec2_instance, from:-7d
| filter isNull(tags) or not iAny(startsWith(tags[], "dt.cost.product:"))
| fieldsKeep id, entity.name, awsVpcName, tags
| sort entity.name asc
| limit 50

// Time range required (corrected 08/12/2026): dt.entity.* is an event-
LOOKBACK view — it returns
// only entities SEEN in the query window, not the standing inventory. Without
an explicit from:
// this counted 5 of 13 EC2 instances against the notebook default window and
looked correct.
// Smartscape note (dt.entity.* is deprecated but still functional): this
cloud resource type
// (EC2 / Azure VM / RDS / Azure SQL / Azure Web App) is not modeled as a
Smartscape node — such
// hosts surface as smartscapeNodes "HOST" with cloud.provider and
aws.*/azure.* fields. Keep the
// classic query above for the cloud-resource inventory.
```

5. AWS Metrics with DQL

EC2 CPU Usage

> **Sources:** timeseries command (DT docs), Semantic dictionary (DT docs) — `dt.host.cpu.usage`, `cloud.aws.lambda.*` metric definitions, AWS topology (DT docs). **Derived:** the `arrayAvg` / `arraySum` post-aggregation pattern is the canonical idiom for sorting a `timeseries` result by total / mean per dimension — see the DQL examples skill for variants.

```
// EC2 CPU usage over the last 6 hours, top 10 busiest instances
timeseries avgCpu = avg(dt.host.cpu.usage), from:-6h, by:{dt.entity.host}
| fieldsAdd avgCpuValue = arrayAvg(avgCpu)
| sort avgCpuValue desc
| limit 10
```

Lambda Invocation Metrics

```
// Lambda invocations over the last 6 hours by function

// Metric keys corrected 08/12/2026. The AWS CloudWatch Lambda metrics are
named
// cloud.aws.lambda.&lt;CloudWatchName&gt;.By.FunctionName — e.g. Invocations
/ Errors / Duration /
// Throttles / ConcurrentExecutions / IteratorAge. The lowercase short forms
used before
// (cloud.aws.lambda.invocations, dt.cloud.aws.lambda.errors, ...) do not
exist, and a timeseries
// against a missing key returns an EMPTY result instead of an error — the
tile just drew nothing.
// The split dimension is FunctionName; dt.entity.aws_lambda_function is null
on these metrics.
// Enumerate what your tenant has with:
//   metrics | filter startsWith(metric.key, "cloud.aws.lambda") | fields
metric.key | sort metric.key asc
timeseries invocations = sum(cloud.aws.lambda.Invocations.By.FunctionName),
from:-6h, by:{FunctionName}
| fieldsAdd totalInvocations = arraySum(invocations)
| sort totalInvocations desc
| limit 10
```

Lambda Error Rate

```
// Lambda error count over the last 6 hours by function

// Metric keys corrected 08/12/2026. The AWS CloudWatch Lambda metrics are
named
// cloud.aws.lambda.&lt;CloudWatchName&gt;.By.FunctionName — e.g. Invocations
/ Errors / Duration /
// Throttles / ConcurrentExecutions / IteratorAge. The lowercase short forms
used before
// (cloud.aws.lambda.invocations, dt.cloud.aws.lambda.errors, ...) do not
exist, and a timeseries
// against a missing key returns an EMPTY result instead of an error — the
tile just drew nothing.
// The split dimension is FunctionName; dt.entity.aws_lambda_function is null
on these metrics.
// Enumerate what your tenant has with:
//   metrics | filter startsWith(metric.key, "cloud.aws.lambda") | fields
metric.key | sort metric.key asc
timeseries errors = sum(cloud.aws.lambda.Errors.By.FunctionName), from:-6h,
by:{FunctionName}
| fieldsAdd totalErrors = arraySum(errors)
| filter totalErrors &gt; 0
| sort totalErrors desc
| limit 10
```

Community Pattern — Lambda Health Score (Errors per Invocation)

Inspired by the Cost Allocation Dashboard (Dynatrace community-examples) pattern of joining two metrics by a shared dimension. Useful as a single-tile signal for SRE dashboards. Community-derived — adapt to your tenant's metric availability.

```
// Lambda error rate (errors/invocations) over 24h by function

// Metric keys corrected 08/12/2026. The AWS CloudWatch Lambda metrics are
named
// cloud.aws.lambda.&lt;CloudWatchName&gt;.By.FunctionName — e.g. Invocations
/ Errors / Duration /
// Throttles / ConcurrentExecutions / IteratorAge. The lowercase short forms
used before
// (cloud.aws.lambda.invocations, dt.cloud.aws.lambda.errors, ...) do not
exist, and a timeseries
// against a missing key returns an EMPTY result instead of an error — the
tile just drew nothing.
// The split dimension is FunctionName; dt.entity.aws_lambda_function is null
on these metrics.
// Enumerate what your tenant has with:
//   metrics | filter startsWith(metric.key, "cloud.aws.lambda") | fields
metric.key | sort metric.key asc
timeseries {
  invocations = sum(cloud.aws.lambda.Invocations.By.FunctionName),
  errors = sum(cloud.aws.lambda.Errors.By.FunctionName)
}, from:-24h, by:{FunctionName}
| fieldsAdd totalInv = arraySum(invocations), totalErr = arraySum(errors)
| filter totalInv &gt; 0
| fieldsAdd errorRatePct = round((totalErr / totalInv) * 100, decimals: 2)
| sort errorRatePct desc
| limit 10
```

6. Metric Availability and Delays

Cloud metrics are not instantly available in Dynatrace. Understanding delays helps set appropriate alerting thresholds.

Source	Typical Delay	Notes
CloudWatch (standard)	5-10 minutes	Free tier, 5-min granularity
CloudWatch (detailed)	1-3 minutes	Additional cost, 1-min granularity
CloudWatch Metric Streams	<1 minute (buffer 60s)	Push-based via Firehose — see §7
ActiveGate polling	5-15 minutes	Depends on number of services/metrics

Implications for Alerting

- **Don't set alert evaluation windows shorter than the metric delay** — a 1-minute evaluation window on a 5-minute metric will produce false negatives
- **Use sliding windows** — evaluate over 15-30 minutes for cloud metrics
- **Combine with OneAgent** — for sub-minute alerting, rely on OneAgent metrics rather than cloud metrics

> **Sources:** Monitor AWS with CloudWatch metrics (DT docs) — polling latency framing, Amazon CloudWatch Metric Streams (DT docs) — push latency and 60s buffer, Amazon CloudWatch concepts (AWS docs) — standard vs detailed monitoring intervals. **Softened:** specific minute-range numbers depend on the number of monitored services, regions, and metric count per polling cycle — use them as planning estimates, not commitments.

7. CloudWatch Metric Streams via Firehose

The Clouds App integration described in §3 is **polling-based** — Dynatrace calls CloudWatch `GetMetricData` on an interval. **CloudWatch Metric Streams** is a **separate, parallel** ingestion path that pushes metrics through Amazon Data Firehose to a Dynatrace HTTP endpoint in near real-time. It is **not** a replacement for the Clouds App — the two coexist.

> **What Metric Streams does NOT replace from the Clouds App:**
> - Entity discovery and topology (the Clouds App still owns `dt.entity.ec2_instance`, `dt.entity.aws_lambda_function`, etc. mapping)
> - AWS tag enrichment, tag-based filtering, and Dynatrace attribute enrichment (`dt.security_context`, `dt.cost.*`)
> - CloudWatch Logs ingestion (Firehose subscriptions configured via the Clouds App — see CLOUD-07)
> - EventBridge event ingestion
>
> Metric Streams handles **metrics only**. For full AWS observability, run Metric Streams *alongside* an existing Clouds App connection.

7.1 When to Add Metric Streams Alongside Clouds App Polling

Driver	Polling alone	Polling + Metric Streams
Latency requirement	5–15 minute alerting acceptable	Need sub-minute metric freshness (e.g., autoscaling reaction, SRE oncall dashboards)
CloudWatch API cost	Manageable at current metric count	`GetMetricData` cost dominates AWS bill; flat Firehose ingest is cheaper
Metric selection granularity	Per-metric / per-statistic control needed	Namespace-level acceptable for the streamed subset
AWS tag-driven workflows	Required (filtering, alerts keyed on tags)	Not needed for the streamed metrics (entity mapping still comes from Clouds App / Hub extension)
ActiveGate footprint	Already deployed	Want to avoid AG capacity scaling for metric polling

Typical pattern: keep the Clouds App polling enabled for entity discovery, tags, logs, and per-metric controls; add Metric Streams for the **high-volume, low-latency-sensitive namespaces** (e.g., `AWS/Lambda`, `AWS/ApplicationELB`, `AWS/RDS`) where polling cost or latency is the constraint.

7.2 Prerequisites

- **API token** in your Dynatrace environment with the **Ingest metrics** permission (note: Metric Streams uses a classic API token, not the Clouds App's Platform Tokens — the two ingest paths authenticate independently)
- **API URL:**
- SaaS — `https://<your_environment_ID>.live.dynatrace.com` or `https://<your_environment_ID>.apps.dynatrace.com`
- ActiveGate — `https://<your_active_gate_IP_or_hostname>:9999/e/<your_environment_ID>`
- **Dynatrace HTTP endpoint** (pick by tenant region): Global `https://aws.cloud.dynatrace.com/`, US `https://us.aws.cloud.dynatrace.com/`, EU `https://eu.aws.cloud.dynatrace.com/`

7.3 Option 1 — CloudFormation (Recommended)

A single CloudFormation stack provisions the IAM role, Firehose delivery stream, CloudWatch Metric Stream, and S3 backup bucket:

```
DYNATRACE_ENV_URL=&lt;your_API_URL&gt;
DYNATRACE_API_KEY=&lt;your_API_token&gt;
STACK_NAME=dynatrace-aws-metric-streams-client
DELIVERY_ENDPOINT=https://aws.cloud.dynatrace.com/
REQUIRE_VALID_CERTIFICATE=true

wget -O dynatrace-aws-metric-streams-client.yaml \
  https://assets.cloud.dynatrace.com/awsmetricstreaming/dynatrace-aws-metric-
streams-client.yaml &amp;&amp; \
aws cloudformation deploy \
  --capabilities CAPABILITY_NAMED_IAM \
  --template-file ./dynatrace-aws-metric-streams-client.yaml \
  --stack-name $STACK_NAME \
  --parameter-overrides \
    DynatraceEnvironmentUrl=$DYNATRACE_ENV_URL \
    DynatraceApiKey=$DYNATRACE_API_KEY \
    RequireValidCertificate=$REQUIRE_VALID_CERTIFICATE \
    FirehoseHttpDeliveryEndpoint=$DELIVERY_ENDPOINT
```

Parameter	Required	Default
`DynatraceEnvironmentUrl`	Yes	—
`DynatraceApiKey`	Yes	—
`STACK_NAME`	Yes	`dynatrace-aws-metric-streams-client`
`RequireValidCertificate`	Optional	`true`
`FirehoseHttpDeliveryEndpoint`	Optional	`https://aws.cloud.dynatrace.com/`

> **Per-region:** Metric Streams are regional. Deploy one stack per AWS region — `export AWS_DEFAULT_REGION=<region>` and re-run.

7.3.1 Alternatives for Restricted-Egress / Security-Conscious Deployments

The `wget` above pulls from `https://assets.cloud.dynatrace.com/awsmetricstreaming/dynatrace-aws-metric-streams-client.yaml`. Many enterprises cannot — or will not — fetch live vendor templates at deploy time:

- Egress proxies, corporate firewalls, or air-gapped AWS accounts block the public asset URL
- Security policy requires every IaC artifact to pass review before it enters the change pipeline
- The pipeline must be fully reproducible — no late-binding fetches from third-party hosts

Five paths handle these constraints. Pick by combining your security posture with your existing tooling.

Path A — Self-host the Dynatrace template (after security review)

The most common enterprise pattern. Fetch the YAML once from any permitted location, **review it**, version it, then deploy from internal copies.

1. **Download** on an approved workstation or jump host:
   `bash wget https://assets.cloud.dynatrace.com/awsmetricstreaming/dynatrace-aws-metric-streams-client.yaml`
2. **Security review** the template before it enters the artifact store. At minimum, verify:
   - IAM roles and policies the template creates (look for any `*` action or `Resource: *`)
   - Resources provisioned (Firehose stream, CloudWatch Metric Stream, S3 backup bucket, IAM roles)
   - Outbound endpoints written into the Firehose `HttpEndpointDestinationConfiguration` (should be the Dynatrace endpoint you expect)
   - No embedded Lambda code, custom resources, or external transformations
3. **Store** the reviewed copy in your internal artifact repo — S3, Artifactory / Nexus, internal Git — tagged with a version (date or SHA256). Treat upstream updates as you would any vendor dependency: review the diff before promoting.
4. **Deploy** from the local copy, with no public-internet dependency at deploy time:
   `bash aws cloudformation deploy \ --capabilities CAPABILITY_NAMED_IAM \ --template-file ./dynatrace-aws-metric-streams-client.yaml \ --stack-name $STACK_NAME \ --parameter-overrides \ DynatraceEnvironmentUrl=$DYNATRACE_ENV_URL \ DynatraceApiKey=$DYNATRACE_API_KEY \`

`RequireValidCertificate=$REQUIRE_VALID_CERTIFICATE \`
`FirehoseHttpDeliveryEndpoint=$DELIVERY_ENDPOINT`

Path B — Build from scratch in your own IaC

If policy forbids vendor templates outright, or you already have a mature Terraform / CDK / Pulumi / in-house CloudFormation pipeline you'd rather extend, build the stack yourself. The Dynatrace template provisions only four resources:

Resource	Purpose
IAM Role (Firehose)	Allows Firehose to assume into AWS, write to the S3 backup bucket, and emit CloudWatch logs
IAM Role (Metric Stream)	Allows CloudWatch Metric Streams to write to the Firehose delivery stream
S3 Bucket	Backup destination for failed deliveries (`FailedDataOnly` mode)
Firehose Delivery Stream	Direct PUT source, HTTP endpoint destination (Dynatrace), GZIP encoding, 3 MiB / 60 s buffer, 900 s retry
CloudWatch Metric Stream	OpenTelemetry 1.0 output, all-or-selected namespaces, writes to the Firehose ARN

A minimal Terraform skeleton (illustrative — adapt to your module conventions):

```
resource "aws_kinesis_firehose_delivery_stream" "dynatrace_metrics" {
  name        = "dynatrace-metric-stream"
  destination = "http_endpoint"

  http_endpoint_configuration {
    url                = "https://aws.cloud.dynatrace.com/"
    name               = "Dynatrace"
    access_key         = var.dynatrace_api_token   # token with metrics.ingest
    buffering_size     = 3
    buffering_interval = 60
    retry_duration     = 900
    s3_backup_mode     = "FailedDataOnly"
    request_configuration {
      content_encoding = "GZIP"
    }
  }

  s3_configuration {
    role_arn   = aws_iam_role.firehose.arn
    bucket_arn = aws_s3_bucket.backup.arn
  }
}

resource "aws_cloudwatch_metric_stream" "to_dynatrace" {
  name          = "dynatrace-stream"
  firehose_arn  = aws_kinesis_firehose_delivery_stream.dynatrace_metrics.arn
  role_arn      = aws_iam_role.metric_stream.arn
  output_format = "opentelemetry1.0"
  # include_filter { namespace = "AWS/Lambda" }  # optional namespace scoping
}
```

You also need the two `aws_iam_role` resources (with trust policies for `firehose.amazonaws.com` and `streams.metrics.cloudwatch.amazonaws.com`), the `aws_s3_bucket` backup target, and CloudWatch log group(s) if you enable Firehose error logging. Cross-check the live Dynatrace template (Path A step 2) periodically for IAM-policy drift — Dynatrace may add scope or change endpoints in newer revisions.

> **Heads-up on the AWS Clouds App side:** there is currently **no Terraform-provider resource for the Dynatrace Clouds App AWS connection itself.** This Path B applies only to the Metric Streams stack, which is independent and fully scriptable.

Path C — Run the original recipe inside AWS CloudShell

AWS CloudShell sessions have outbound internet enabled by default. If the egress restriction is on your laptop / corporate VPN but not on the AWS account itself, open CloudShell from the AWS Console and run the §7.3 recipe verbatim — no template hosting needed, one-off per region. Some highly regulated AWS accounts disable CloudShell or restrict its outbound network — confirm with your AWS administrator.

Path D — Manual Console setup (no template at all)

The §7.4 manual setup (Firehose Delivery Stream → CloudWatch Metric Stream) requires no template fetch. Slower for multi-region rollout, but unblocks fully air-gapped accounts and shops with strict "no third-party IaC" policies. A documented runbook + screenshots makes it repeatable without making it scriptable.

Path E — Request a signed / checksummed artifact via your Dynatrace account team

Some procurement teams require vendor templates to arrive through an auditable channel rather than a public asset URL. Worth asking your Dynatrace account team whether they can provide:

- A SHA256 checksum for the published template (so you can verify the copy you fetched matches the published version)
- A signed / versioned release artifact
- Access via a private artifact feed your account is entitled to

This is an account-team conversation, not a self-service option — but for highly regulated industries (financial services, government, healthcare) the conversation is often necessary anyway.

Scenario → recommended path

Scenario	Best path	Why
Has IaC pipeline + standard security-review process	**Path A**	Fastest; review-once-then-deploy fits typical change-management
Existing Terraform / CDK estate, prefers full control	**Path B**	Native fit; no vendor-template review every release
Egress blocked at laptop/VPN but AWS account is fine	**Path C**	Zero setup; quickest path through a corporate-network constraint
No IaC allowed from external sources at all	**Path D**	Pure console; pair with a runbook for repeatability

Scenario	Best path	Why
Procurement / compliance requires signed artifacts	**Path E**	The auditable supply-chain conversation

> **Sources:** Amazon CloudWatch Metric Streams (DT docs), aws_cloudwatch_metric_stream (Terraform AWS provider), aws_kinesis_firehose_delivery_stream (Terraform AWS provider), AWS CloudShell User Guide. **Derived:** the 5-path matrix and scenario→path table synthesize public-docs primitives with field-observed enterprise procurement patterns — no single source endorses this composition by name. **Softened:** the "ask your Dynatrace account team" Path E is industry-practice guidance, not a documented Dynatrace offering — confirm what your account is entitled to before committing the path to a runbook.

7.4 Option 2 — Manual Setup in AWS Console

Step 1 — Firehose Delivery Stream

1. **Kinesis** → **Create delivery stream** · Source: **Direct PUT** · Destination: **Dynatrace**
2. Disable **Data Transformation**
3. Destination settings: Ingestion type **Metrics**, HTTP endpoint URL (Global/EU/US), API token + API URL, content encoding **GZIP**, retry duration `900`, buffer hints **3 MiB / 60 s**, backup **Failed data only** → new S3 bucket
4. **Create delivery stream**

Step 2 — CloudWatch Metric Stream

1. **CloudWatch** → **Metrics** → **Streams** → **Create metric stream**
2. **Custom setup with Firehose** → enter the Firehose name
3. **Change output format** → **OpenTelemetry 1.0**
4. Metrics scope: **All metrics** or **Select metrics** (namespace-level only)
5. Name and **Create**

7.5 Post-Setup — Enable the AWS Entities Hub Extension

In Dynatrace, open **Hub** and enable **AWS Entities for Metric Streaming**. Without this extension, streamed metrics arrive in Grail but are not attached to entity types — DQL

queries using `by:{dt.entity.ec2_instance}` etc. on streamed metrics will return empty. Note that even with the extension enabled, AWS resource **tags** are not propagated onto streamed metrics — for tag dimensions, query the same metric via the Clouds App polling path.

7.6 Gotchas

Caveat	Implication
Parallel to Clouds App, not a replacement	Disabling Clouds App metric polling on services you also stream loses tag enrichment and per-metric controls. Decide deliberately which path owns each namespace.
No AWS tag propagation on streamed metrics	Filter by tag is unavailable for the streamed subset. Workflow: stream high-volume namespaces, keep polling on tag-driven services.
Namespace-level metric selection only	All-or-nothing per AWS service in the stream — you cannot exclude individual metrics within a namespace.
Silent delivery failures	Firehose delivery failures do **not** generate Dynatrace alerts. Monitor the S3 backup bucket and configure a CloudWatch alarm on Firehose `DeliveryToHttpEndpoint.Success`.
Metric age limit	Metrics older than 2 hours are dropped — backfills won't replay.
Per-region cost	Each region adds Firehose ingestion + S3 backup storage. Plan regional scope deliberately.
No Terraform support for the Clouds App connection itself	The CFN-based Clouds App onboarding has no Terraform-provider resource as of this writing. Metric Streams is a separate stack and remains scriptable via plain CloudFormation.

> **Sources:** Amazon CloudWatch Metric Streams (DT docs), Monitor AWS with CloudWatch metrics (DT docs), Create AWS connection via Settings (DT docs). **Derived:** the §7.1 "polling + selective streaming" coexistence pattern synthesizes both pages — no single page recommends this composition by name. **Softened:** the namespace-shortlist for streaming (Lambda / ELB / RDS) is community guidance; pick based on your own polling cost and latency requirements.

8. Cost Optimization

AWS CloudWatch API calls are billed per request. Dynatrace cloud integration generates API calls for:
- **GetMetricData** — retrieving metric values
- **DescribeInstances** / **ListFunctions** — entity discovery
- **ListMetrics** — metric enumeration

Cost Reduction Strategies

Strategy	Impact	How
Disable unused services	High	Only enable services you actively monitor
Tag-based filtering	High	Monitor only `dynatrace-monitored: true` resources
Reduce metric count	Medium	Disable individual metrics you don't need
Use Metric Streams	Medium	Push-based; eliminates `GetMetricData` poll cost — see §7
Regional scoping	Medium	Only monitor regions where workloads run

Estimating CloudWatch API Costs

Approximate formula:

```
Monthly API calls = (services enabled) x (metrics per service) x (resources) x (polls per day) x 30
Monthly cost = API calls / 1000 x $0.01
```

For example: 5 services x 10 metrics x 100 resources x 288 polls/day x 30 = ~43M calls/month = ~$430/month

> **Tip:** Review the CloudWatch billing dashboard monthly and correlate spikes with Dynatrace service additions.
>
> **Sources:** Amazon CloudWatch pricing (AWS docs) — `GetMetricData` request pricing tier and `$0.01 per 1,000 metrics requested` (verify current rate per region; the example formula

> assumes a US region rate), Monitor AWS with CloudWatch metrics (DT docs), Tags and management zones for AWS (DT docs). **Derived:** the cost-reduction table prioritization (disable-unused > tag-filtering > metric-count > streams > regional-scope) is field-observed guidance based on which levers have the largest absolute impact per typical enterprise tenant. **Softened:** the worked monthly cost example uses representative round numbers; recompute against your tenant's actual service/resource counts and current AWS pricing before quoting to stakeholders.

9. Related Resources

Resources are grouped by source tier — official Dynatrace docs are authoritative; official Dynatrace open-source repos are maintained but version-pinned to specific Dynatrace releases; community examples are reference implementations, not production artifacts.

Official Dynatrace Documentation

Primary references for everything in this notebook. See REFERENCE.md for the full bibliography.

- AWS Cloud Platform Monitoring — the canonical onboarding entry point
- Manage your AWS connections — post-onboarding lifecycle
- All AWS cloud services — current monitored-service catalog (changes per release)

Official Dynatrace Open-Source (Secondary)

Maintained by Dynatrace on GitHub. Pin to a release before deploying to production — `main` is a moving target.

- Dynatrace Operator (dynatrace-operator) — DynaKube CRD + Kubernetes Operator (DT-published, last verified 06/05/2026). The route for EKS / AWS-hosted Kubernetes — see CLOUD-03.
- S3 Log Forwarder (dynatrace-aws-s3-log-forwarder) — Serverless application pushing logs from Amazon S3 to Dynatrace (DT-published, last verified 06/05/2026). Use for log sources written to S3 by AWS services that don't write to CloudWatch Logs directly.

> **Disclaimer — Lambda extension distribution:** Dynatrace's AWS Lambda monitoring is distributed via AWS Lambda Layers and the AWS Serverless

> Application Repository (SAR), not as a single open-source repo. The previously-archived `dt-awslayertool` (2021) is not a current artifact. Follow the canonical Lambda docs in CLOUD-04 rather than searching GitHub.

Community Examples (Tertiary — Adapt, Don't Adopt)

⚠️ **Disclaimer:** These are community-curated examples in the Dynatrace/community-examples repository. They are Dynatrace-published but explicitly marked as community contributions — not officially supported, not part of the product, and may lag behind current Dynatrace features. Verify against the latest DQL syntax and your tenant's schema before adopting. The repo is actively maintained (last verified 06/05/2026).

- Cost Allocation Dashboard (community) — DPS cost allocation patterns; the metric-join pattern in §5's Lambda Health Score is inspired by it
- DQL Usage Overview (community) — DQL execution cost monitoring; useful for understanding which AWS queries are expensive
- CI-CD Pipeline App (community) — pipeline observability via OpenPipeline; integrates with AWS CodePipeline / CodeBuild signals

AWS-Side References (Primary, External)

- Amazon CloudWatch concepts (AWS docs) — standard vs detailed monitoring, metric resolution
- AWS Well-Architected Framework — Operational Excellence Pillar — observability and telemetry guidance; pair with this notebook for governance reviews
- AWS Cost Categories (AWS docs) — the AWS-side primitive for the `dt.cost.product` mapping pattern in §3

> **Sources:** all links above are primary or Dynatrace-published. Recency verified 06/05/2026 against each repo's last-pushed date and each docs page's last-published indicator. **Derived:** the three-tier framing (DT docs / DT GitHub / community) is this notebook's editorial convention per `.claude/rules/code-style.md` § 6, not a Dynatrace-published tiering.

10. Summary and Next Steps

Key Takeaways

- Use **IAM role-based authentication** for production AWS integrations
- Consider adding **CloudWatch Metric Streams via Firehose** *alongside* Clouds App polling for high-volume / low-latency-sensitive namespaces (e.g., Lambda, ELB, RDS) — Streams handles metrics only and does not replace the Clouds App
- Enable services in **tiers** based on business criticality, not all at once
- **Tag-based filtering** is the most effective cost optimization lever
- Account for **metric delays** (5-15 minutes) when configuring alerts
- Combine **cloud integration** with **OneAgent** for full-stack visibility

Next Steps

- **CLOUD-03: AWS EKS Monitoring** — Deep dive into Kubernetes on AWS
- **CLOUD-04: AWS Lambda & Serverless** — Serverless function monitoring
- **CLOUD-07: CloudWatch Log Ingestion** — Log forwarding from AWS
